

Intermediate (defend one non-obvious trade-off under pushback) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order against your own design before reading ahead. Each hint has 2 depth levels: **Basic** (the plain framing — how to pick a decision and find a real objection) and **Intermediate** (the real structure a defense needs, then the senior-level judgment call — defending it under a second, harder round of pushback, not just the first objection). Read Basic first even if you're confident — it's the fastest way to see exactly what each deeper level adds.

- [Hint 1 — The idea, and finding a real objection](#)
- [Hint 2 — The plan, and a worked defense](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and finding a real objection

Basic Version

Pick a decision from your design that has a real, sensible alternative — not something that's obviously correct. "I used LangGraph" isn't debatable if the whole curriculum used LangGraph. "I gave the supervisor full control over routing instead of a fixed sequence" is debatable — a fixed sequence is simpler, and simpler is often better.

Once you've picked one, write the strongest case *against* your own choice before you write your defense of it. This is the part people skip, and it's the part that actually makes the exercise work.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Doc16's framing of "defending a design when it's pushed on" applies here directly: the goal isn't to win the argument, it's to show you *actually weighed* the alternative, not just picked the first option you thought of. A defense that just restates the choice ("I did X because X is good") isn't a defense — it's a preference.

Structure: **the decision** → **the strongest real alternative** → **what that alternative would have cost you, specifically, for THIS system's requirements** → **your actual reasoning for choosing what you chose**. The word "specifically" matters — a generic cost ("it's more complex") is weaker than a cost tied to your actual requirements ("it's more complex, and this system has a 3-person team maintaining it, so that complexity has a real maintenance cost we can't absorb").

Common categories of real objections, if you're stuck finding one: **simplicity** ("why not skip this entirely?"), **cost** ("doesn't this add latency/dollars you don't need yet?"), **failure mode** ("what happens when this specific piece breaks?"), **maintainability** ("who maintains this after you're gone, and is it harder than the alternative?").

Write out your decision and its strongest counter-argument now, before Hint 2.

When the pushback comes back a second time:

A defense that survives exactly one round of pushback and then stops isn't fully tested — a real senior reviewer doesn't ask one question and move on. The harder, more realistic version: after you answer the first objection, the reviewer attacks your *mitigation* instead of your original decision. You said "I accepted risk X, but I mitigate it with Y" — the harder pushback is "fine, but Y itself has a hole. What then?"

This is a different skill than the first round. The first round tests whether you weighed a real alternative. The second round tests whether your answer was a genuine, load-bearing part of the design, or just a reassuring sentence you added to sound thorough. If Y turns out to be as weak as the reviewer suggests, the honest move is to say so and describe what you'd actually add — not to invent a second, equally thin justification on the spot to avoid conceding anything.

A second, related curveball worth rehearsing: the reviewer names a *different* decision elsewhere in your design that's in tension with the one you just defended — forcing you to notice and admit an actual inconsistency, rather than defending both halves as if they were never in conflict.

Try defending your chosen decision through two full rounds of pushback — first on the decision, then on your own mitigation — before checking Hint 2.

Difference between Basic and Intermediate: Basic states the decision, the strongest objection, and an honest answer. Intermediate names what the alternative would really have cost and the mitigation you built — and then holds up when the reviewer attacks the mitigation itself, or finds a second decision that conflicts with the first.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and a worked defense

Basic Version

pick a real, debatable decision from your design

write the strongest objection to it (not a weak one)

answer:

what would the alternative have actually cost me, for THIS system?

what does my choice cost me, that I'm accepting on purpose?

would I actually change my mind if pushed harder? if yes, say so.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Decision: state it in one sentence, plainly.

Strongest objection: state the alternative and its real advantage – don't strawman it. If the objection is genuinely strong, say so before defending against it.

Cost of the alternative, specific to THIS system: name a concrete consequence tied to an actual requirement (scale, team size, cost cap, reliability target) – not a generic downside.

Cost of your own choice, accepted on purpose: name what you gave up. A defense with no acknowledged cost is not a real defense.

Verdict: would you actually change your mind under harder pushback? If genuinely yes – say so, and explain what evidence would flip you.

Worked example, partially filled in:

Decision: "I used a supervisor agent instead of a fixed pipeline."

Strongest objection: "a fixed pipeline is simpler to read and debug – why add a supervisor's routing complexity if you don't strictly need it?"

Answer: ____ (fill in: what does your system specifically need that a fixed pipeline can't do?)

Fill in your own answer, then plan for the second round of pushback below.

Round 2, as a plan:

The same decision, taken through a second round — the reviewer doesn't accept your mitigation and pushes again:

Round 1 decision: "I gave every agent access to most of the shared state, instead of giving each agent only its own private slice."

Round 1 objection: "shared mutable state is a known source of

hard-to-trace bugs, and it gets worse as agent count grows."

Round 1 answer: "fully private state doesn't remove the risk, it just moves it -- from 'read the wrong shared field' to 'forget to pass the right field forward,' which fails silently. I mitigate the real risk with a schema check at each node boundary: each agent's node function only writes to its own declared output keys."

Round 2 (the harder pushback -- attacks the mitigation, not the decision): "okay, but a write-boundary schema check only catches WRITES to the wrong field. It does nothing about an agent silently READING a field it has no business depending on -- say, the writer reading the fact-checker's private scratch notes and being subtly influenced by them in a way nobody intended. Your mitigation doesn't cover that at all. What then?"

Round 2 answer: fill this in yourself before checking the Solution -- does the schema check actually cover this? If not, what would you honestly add, or is read-access control a cost you'd knowingly accept and explain why?

Write your own Round 2 answer, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic plans one round of pushback. Intermediate plans for the second round too: separating what your fix covers from what it doesn't, and conceding a real gap with a prioritisation reason instead of inventing a justification.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)